1 Modelowanie i reprezentacja informacji (Syntaktyka i Semantyka)

Kluczowym wyzwaniem w rozwoju naszego zintegrowanego systemu wizualnego na Wydziale MiNI jest nie tylko wyznaczanie badanych wcześniej tras, ale przede wszystkim dostarczanie wiedzy na żywo o dostępności poszczególnych stref (np. *Pokój cichej nauki*, *Strefa Jet Brains*) bezpośrednio na wielkoformatowe ekrany w lobby. Aby było to możliwe, system musi nieustannie dokonywać zamiany fizycznego świata rzeczywistego na użyteczną postać cyfrową. Realizujemy to dzieląc zjawisko informacji na dwa zasadnicze poziomy: syntaktyczny (zbieranie struktury sygnałów) oraz semantyczny (nadawanie im ostatecznego znaczenia).

1.1 Poziom syntaktyczny: Zbieranie surowych danych

Na poziomie syntaktycznym system nie rozumie kontekstu zdarzeń na wydziale – zajmuje się jedynie automatyczną rejestracją surowej formy sygnałów fizycznych. Z uwagi na restrykcyjne normy prywatności i przepisy o ochronie danych (RODO), w projekcie całkowicie odrzuciliśmy możliwość korzystania z kamer wideo oraz jakichkolwiek mikrofonów. Podstawą naszego systemu stały się w pełni zanonimizowane urządzenia pomiarowe. Zbieramy następujące typy strumieni danych:

- **Czujniki obecności (sieć ZigBee[1]):** Gotowe radary milimetrowe (mmWave) podpięte pod sufity naszych sal. Zamiast tworzyć skomplikowaną obróbkę samych fal radiowych na backendzie, zrzucamy pracę na wbudowane oprogramowanie radaru. Otrzymujemy z niego po prostu surową liczbę typu `integer` (całkowitą), która mówi wprost: "widzę w tej sali fizycznie X osób".
- **System kontroli wejść na legitymacje (RFID/NFC):** Klasyczne czytniki zbliżeniowe montowane przed wejściem do określonych stref wydziału (takich jak *Pokój cichej nauki*). Każde odbicie fizycznej legitymacji generuje po prostu standardowy strumień systemowych zapytań dodających/odejmujących studenta z tablicy w bazie danych.

Z perspektywy prawa oświatowego i braku konieczności instalowania kamer – układ ten sprawdza się doskonale. Jednakże dla końcowego odbiorcy – czyli studenta w holu – goła liczba logowań z czytników czy surowe zera z ZigBee to na ten moment bezwartościowy szum (fizyczna syntaktyka). Wyświetlenie takich surowych i nieprzetworzonych porcji zmiennych na wielkich ekranach byłoby dla studentów po prostu brzydkie i stanowczo niezrozumiałe.

1.2 Poziom semantyczny: Interpretacja i sens informacji

Aby odciążyć odbiorcę z nadmiarowości i dostarczyć wartość, dane muszą przejść cyfrową transformację na poziom semantyczny. Tu nasz autorski algorytm łączy i przetwarza surowe ciągi w konkretną wiedzę, dokonując dogłębnej interpretacji w pełni uwzględniającej anonimowość.

[1] ZigBee to popularny i wysoce energooszczędny standard łączności bezprzewodowej, powszechnie stosowany w branży Smart Home. Pozwala on łatwo i bezpiecznie połączyć wszystkie sensory na piętrze w jedną, wspólną i stabilną sieć komunikacyjną.

W naszym projekcie stawiamy na bardzo proste, programistyczne połączenie odczytów z obu urządzeń. Na poziomie semantycznym, zadaniem naszego oprogramowania jest po prostu zestawienie ze sobą surowych liczb i wywnioskowanie z nich jednego, pewnego wyniku dla konkretnej sali.

Działa to w naszej logice następująco: radar podwieszony nad małym pokojem do nauki (o pojemności 6 krzeseł) przesyła nam na serwer informację po API, że widzi fizycznie 4 osoby. Z kolei z bazy czytnika kart wynika, że wejście "odbiły"tylko 3 osoby. Nasz algorytm zawsze bezpiecznie wybiera większą wartość z tych dwóch źródeł, żeby się ubezpieczyć przed błędem sprzętowym (w tym przypadku zakładamy, że jeden student najzwyczajniej wszedł za kolegą bez przykładania własnej legitymacji do drzwi). Radar mądrze doliczył nam więc "brakującego"człowieka.

Mając wiarygodną wartość, system musi nadać jej jeszcze proste znaczenie (czyli przejść do warstwy semantycznej w 100%). Skrypt analizuje, że skoro sala ma wpisany stały limit 6 wolnych miejsc, to oprogramowanie w mgnieniu oka redukuje całą tę skomplikowaną dedukcję do czytelnego, graficznego komunikatu. Na wielkim ekranie w lobby zapala się prosty napis: **„Zajętość strefy: 4/6 miejsc”**.

Z perspektywy teorii i inżynierii danych określilibyśmy to klasycznym przeskokiem wymiarowości. Za kulisami ukrywamy zapytania do bazy języka SQL, wyliczany ruch milimetrowy Zigbee z opóźnieniami sieciowymi, a docelowo serwujemy studentowi na dole jasny wynik ułamkowy 4/6. Rzut oka na taki czysty kafel w systemie i nasz statystyczny odbiorca w sekundę podejmie decyzję, co robi dalej bez marnowania przerw między zajęciami na daremtne wycieczki po uczelni.

1.3 Struktury wiedzy: Taksonomia stanów pokoju

Żeby wyliczone obłożenie ułamkowe wyświetlało się w sensowny i szybki sposób dla studentów, podzieliliśmy wszystkie pokoje na trzy proste statusy. Aplikacja na serwerze przydziela je u dołu ekranów automatycznie, dając na mapie odpowiedni kolor kafelka:

1. **WOLNY (Zielony):** W sali jest idealnie pusto. Algorytm wyciągnął ostatecznie twarde "0"i z czujników ruchu i z logowań w drzwiach. Na mapie wydziału sala świeci się na zielono, oznaczając pełną swobodę.

2. **ZAJĘTY (Czerwony):** Z krzyżowej weryfikacji sprzętowej wynika, że fizycznie do pokoju weszło bardzo dużo osób (obłożenie ułamkowe obliczone na ponad 90%). Miejscówka dostaje na ekranach czerwoną flagę i staje się wirtualnie niedostępna, co z wyprzedzeniem zniechęca studentów do podejmowania wędrówki w tym kierunku.

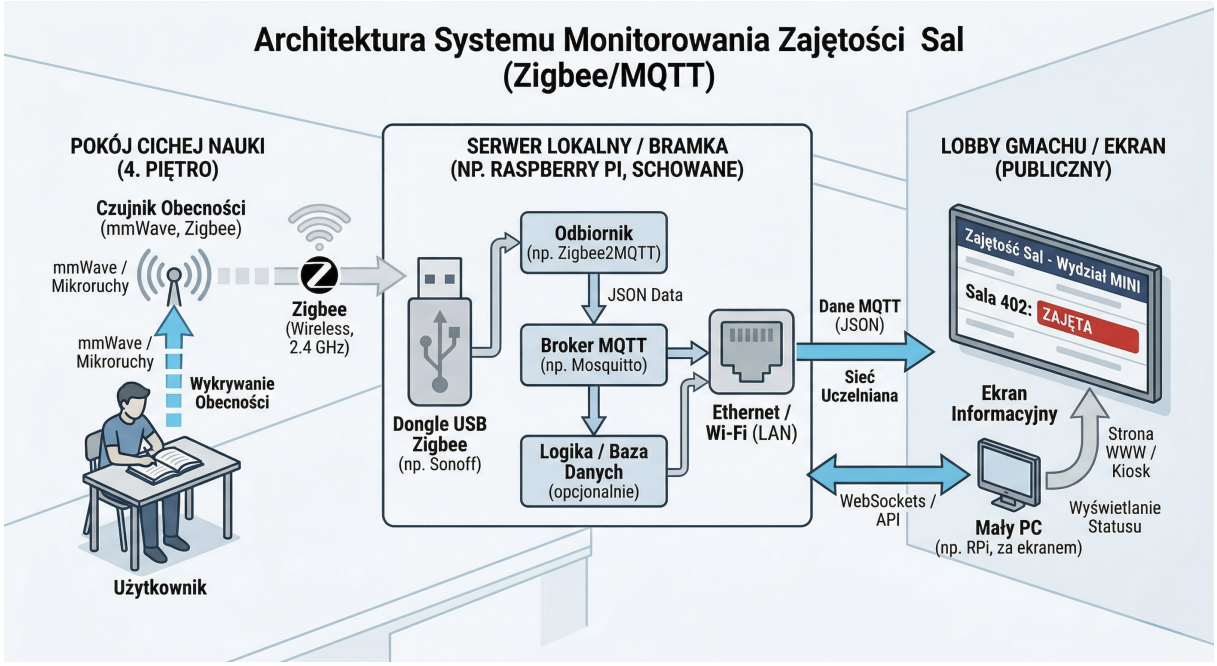
3. **SPRZĄTANIE (Szary):** Mechanizm twardego serwisu o najwyższych uprawnieniach systemowych w hierarchii. Gdy pracownik techniczny odbija na czytniku swoją autoryzacyjną Kartę Master, nadpisuje to z automatu całą inteligentną detekcję danego pomieszczenia. Pokój zniknie z grafu operacyjnego dla całego wydziału aż do momentu ponownego zwolnienia karty po wyjściu obsługi.

Podsumowując, nasz unikalny system podejmuje długą i solidną drogę programistyczną. Zaczynamy od hardware'owych detektorów pod sufitem ograniczających problematykę RODO (syntaktyka). Następnie zestawiamy je z wejściami legitymacji i wyliczamy ułamek zajętości miejsc, ubezpieczając się na wypadek wejścia kogoś niejawnie (semantyka). Wrzucenie tego w końcową, kolorową listę statusów stanowi piękny dowód na to, jak

z tysiąca odczytów radiowych uzyskać prostą, zielono-czerwoną tablicę dla zabieganego studenta przed wykładem.

1.4 Architektura przesyłu danych

Aby system tak płynnie przechodził od czujników do ostatecznej oceny zajętości na wyświetlaczach w holu, korzystamy ze świetnie zaplanowanej architektury technologicznej. Poniższy obraz przedstawia najważniejsze moduły tego układu:



Rysunek 1: Architektura Systemu Monitorowania Zajętości Sal w technologii Zigbee i MQTT.

Działanie całości jest bardzo proste i można je opisać w zaledwie trzech krokach przepływu informacji:

- **1. Środowisko fizyczne (Pokój Nauki - lewa strona):** Czujnik (rozpoznający ruch fizyczny z radaru) wykrywa, że student siedzi przy biurku. Ponieważ używa niezwykle energooszczędnej technologii Zigbee, wysyła wynik detekcji w powietrze drogą bezprzewodową na częstotliwości 2.4 GHz.
- **2. Hub Główny (Serwer Ukryty - środek):** Gdzieś dalej w budynku schowany jest malutki mikrokomputer nadzorczy (np. tanie, małe Raspberry Pi). Posiada on wetknięty do gniazda "gwizdek"USB (dongle przypominający pendrive), który świetnie radzi sobie z przechwytywaniem tych sygnałów z powietrza. Kiedy komputer je wyłapie, specjalne programy narzędziowe (np. protokół MQTT) tłumaczą surowy szum antenowy na klasyczny, ludzki tekst komputerowy typu "stan: obecność"i wgrywają to do głównej, twardej sieci ethernetowej uczelni.
- **3. Klient Końcowy (Z publicznym ekranem - prawa strona):** Sam wielki telewizor w holu wyposażony jest po prostu w kolejny minikomputer stykający się ze stroną WWW. Ten komputer w holu nie przeprowadza detekcji ani żadnych

skomplikowanych obliczeń radarów i anten – jest stacją wyjściową. Przez internet co chwilę komunikuje się dyskretnie z Serwerem Ukrytym i na podstawie gotowego już wyniku po prostu maluje duży pasujący kafelek statusu ("Sala: Zajęta") prosto w oczy mijających go potoków studentów.

Rozbicie fizyki na te trzy elementy sprzętowe mocno upraszcza pisanie każdej linijki kodu oraz pozwala np. zresetować duży ekran w holu bez przerywania analizy liczenia wejściówek do stref nauki.